関数表

Author: yuichi(作成 2026-10-21)

CsqWatch のスクリプトと画面に含まれる関数の一覧です。設計の考え方は architecture.md を参照してください。

ファイルの関係

```
Monitor.bat ──► Monitor.ps1 ─┬─ . Config.ps1      (設定)
                             ├─ . Common.ps1      (Invoke-At → AtIo.exe)
                             └─ . Browser.ps1     (Open-Dashboard → ブラウザ)
Probe.bat   ──► Probe.ps1   ─┬─ . Config.ps1
                             └─ . Common.ps1
index.html  ──► data.js (Monitor.ps1 が書き出す)
build.bat   ──► AtIo.cs → AtIo.exe
```

. ファイル名 は PowerShell のドットソース(同じスコープに読み込む)です。

Monitor.ps1(収集の本体)

関数	引数	戻り値	役割	呼び出し元
`Write-TestLog`	`Level` (区分)、`Msg` (内容)	なし	試験ログ `logs\test_YYYYMMDD.log` に 1 行追記する。日時はミリ秒まで。内容の改行は `|` に置き換える。書き込みの失敗は無視する(監視を止めない)	ほぼすべて
`Show-Dashboard`	なし	なし	`Open-Dashboard` を呼び、結果を試験ログに書く。開けなければコンソールに案内を出す	起動時、二重起動時
`Test-InnerException`	`Ex` (例外)、`Type` (型)	真偽値	例外とその内側の例外をたどり、指定の型が含まれるかを調べる。Mutex の放棄 (`AbandonedMutexException`)の判定に使う	二重起動の確認
`Format-Response`	`Res` (応答)	文字列	ログ用に応答を整える。PowerShell 5.1 の「発生場所 …」などの飾りと `AtIo.exe :` の前置きを除く。AT の応答(`+CSQ:` など)は残す。空なら `(空応答)`	`Invoke-AtLog` 、`Invoke-Poll`
`Invoke-AtLog`	`Cmd` 、`Wait` (秒)、`After` (後続コマンド、省略可)	応答(整形前)	`Invoke-At` を包み、送信(`CMD`)・応答と所要時間(`RSP`)を試験ログに残す。`AtIo.exe` が残っていれば `WARN`	`Invoke-Poll` 、`Invoke-TestCall`
`Get-CsvPath`	`d` (日時)	パス	その月の CSV のパス `logs\csq_YYYYMM.csv` を返す	`Add-CsvLine` 、`Import-History`
`Add-CsvLine`	`d` (日時)、`line` (1 行)	なし	CSV に 1 行追記する。ファイルがなければ見出し付きで作る。失敗したら(Excel で開いている場合など)警告を出し、試験ログに `ERR`	`Invoke-Poll` 、`Invoke-TestCall`
`Import-History`	なし	なし	起動時に、その月の CSV の末尾 200 行からグラフの履歴を復元する。新しい CSV がなければ旧名 `isat_YYYYMM.csv` を読む。3 列目が `N/A` ・ `取得失敗` の行は取得失敗として扱う	起動時
`Write-Data`	なし	なし	画面用の `data.js` を書き出す(一時ファイルに書いてから置き換え)。履歴は、取得失敗を `null` 、端末の圏外(99)を `0` にする	起動時、`Invoke-Poll`
`Invoke-Poll`	なし	なし	1 回分の取得。COM ポート一覧の確認 → `AT+CSQ` → `AT+CREG?` → `$Extras` の順に送り、解析して CSV・試験ログ・ `data.js` に反映し、コンソールに 1 行表示する	メインループ
`Invoke-TestCall`	`now` (日時)	なし	定時試験発信。 `ATD<番号>;` を送り、`$CallDuration` 秒後に `ATH` で切る。応答を試験ログに、発信の記録を CSV に残す	メインループ

Monitor.ps1 の関数以外の処理(上から順に)

処理	内容
設定の読み込み	`Config.ps1` がなければ `Config.sample.ps1` からコピー。`Config.ps1` ・ `Common.ps1` を読み込む。`$RssiMax` ・ `$DeviceName` ・ `$AtIoExe` が未設定なら既定値を入れる
`trap`	予期しないエラーで止まる直前に `ERR UNHANDLED` (行番号付き)を試験ログに残す
二重起動の確認	Mutex `Global\CsqWatch` を取る。取れなければ監視画面を前面に出し、5 秒後に終了コード `3` で終わる
起動の準備	`monitor_running.txt` を書く。窓のタイトルを「CsqWatch 稼働中」にする。CPU 優先度を「通常以下」にする
起動記録	`START` (PID・PowerShell の版・PC の起動時刻)と設定値を試験ログに残す
メインループ	1 秒ごとに時刻を確認し、定時発信と周期取得を行う。周期が 10 秒を超えて遅れたら `WARN` (スリープの兆候)
`finally`	止めたとき(Ctrl+C を含む)に `STOP` を残し、`monitor_running.txt` を消し、Mutex を解放する

Common.ps1(AT コマンド送信の共通部)

関数	引数	戻り値	役割	呼び出し元
`Invoke-At`	`Cmd` 、`Wait` (秒、既定 3)、`After` (省略可)	応答の文字列	`AtIo.exe <Port> <Baud> <Cmd> <Wait> [After]` を呼び、標準出力と標準エラー出力をまとめて返す。`-Mock` のときは `Get-MockResponse` を返す。呼び出しに失敗したら `IO Error: …` を返す	`Invoke-AtLog` 、`Probe.ps1`
`Get-MockResponse`	`Cmd`	応答の文字列	`-Mock` 用の疑似応答。`AT+CSQ` は rssi 20〜50(IsatPhone 2 の実測付近)でゆらぐ値、`AT+CREG?` は rssi が 25 未満なら探索中。ほかに `AT+COPS?` ・ `AT+CBC` (実機の IsatPhone 2 では使用禁止)・ `AT` ・ `ATH` 。それ以外は `ERROR`	`Invoke-At`

Browser.ps1(監視画面を開く)

関数	引数	戻り値	役割	呼び出し元
`Find-BrowserExe`	`Name` (`chrome` / `edge` / `firefox`)	パス、なければ `$null`	レジストリの App Paths と標準のインストール先から、ブラウザの実行ファイルを探す	`Open-Dashboard`
`Initialize-FirefoxProfile`	`Prof` (プロファイルのフォルダ)	なし	Firefox の専用プロファイルを作り、初回の案内などを出さない設定 (`user.js`)を書く	`Open-Dashboard`
`Find-DashboardProcess`	`Exe` 、 `Prof`	プロセス ID、なければ `$null`	専用プロファイルで動いているブラウザの本体プロセスを探す(コマンドラインにプロファイルのパスを含み、子プロセスではないもの)。社外監視のブラウザは対象にならない	`Open-Dashboard`
`Open-Dashboard`	`Dir` 、 `Browser` (既定 `auto`)、 `Size` 、 `Pos`	`OK …` または `NG …`	ブラウザを選び、すでに開いていればそのプロセスの窓を前面に出す (`AppActivate`)。なければ専用プロファイル・アプリモードで `index.html` を開く	`Show-Dashboard`

Probe.ps1(対応コマンドの調査)

関数はありません。上から順に次の処理を行います。

処理	内容
準備	`Config.ps1` ・ `Common.ps1` を読み込み、 `probe_commands.txt` (または `-ListFile`)から `#` で始まらない行を読む
送信と判定	1 コマンドずつ `Invoke-At` (待ち 5 秒)で送り、 `NO RESPONSE` / `IO ERROR` / `OK` / `ERROR` / `OTHER` に分ける
保存	生の応答と一覧を `probe_YYYYMMDD_HHMMSS.txt` (UTF-8 BOM 付き)に保存する

Atlo.cs(シリアル I/O、C#)

メソッド・定義	役割
`Main(string[] args)`	引数を読み、COM ポートを 8N1・DTR/RTS ON で開く。 `ATE0V1` を送って応答を捨て、コマンドを送り、受信した応答をそのまま標準出力へ出す。終了コード `0` 正常 / `1` 引数不足 / `2` I/O エラー
`IsHelp(string a)`	`-HELP` / `--HELP` / `-H` / `-?` / `/?` / `/HELP` かどうか(大文字小文字は区別しない)
`Final` (正規表現)	応答の終端行(`OK` / `ERROR` / `NO CARRIER` / `BUSY` / `NO ANSWER` / `NO DIALTONE` / `+CME ERROR:` / `+CMS ERROR:`)を、受信済みの全文から探す
`HelpText` (定数)	`-HELP` で表示する使い方

index.html(画面、JavaScript)

関数	役割
`$(id)`	`document.getElementById` の短縮
`setBadge(text, cls)`	右上の状態表示(`LIVE` / `STALE` / `NO DATA` など)を変える
`initChart()`	グラフ(Chart.js)を作り、 `loadData` を 3 秒ごとに呼ぶように設定する
`loadData()`	`<script src="data.js?t=…">` を差し替えて読み込み、読めたら `render` を呼ぶ。読めなければ `NO DATA`
`render(d)`	`window.CSQ_DATA` からカード・見出し(機器名)・グラフを更新する。取得失敗は `--` と赤の `取得失敗` 、端末の 99 は `圏外 (99)` 。グラフの縦軸は `rssiMax` に合わせる。最終更新が周期の 2 倍 + 10 秒より古ければ `STALE`

バッチファイル

ファイル	内容
`Monitor.bat`	`Monitor.ps1` を起動する(引数はそのまま渡す)。終了コード `3` (すでに動いている)のときは `pause` せずに閉じる
`MonitorMock.bat`	`Monitor.ps1 -Mock` を起動する。終了コード `3` の扱いは `Monitor.bat` と同じ
`Probe.bat`	`Probe.ps1` を起動する(引数はそのまま渡す)
`build.bat`	Windows 標準の `csc.exe` (64bit を優先)で `AtIo.cs` から `AtIo.exe` を作る